Future Generation Computer Systems 00 (2027) 1–7

Who Should Look at This Image? Small Language Agents Choosing the Tools of Inspection Drones

Mahdi Khashan*

[a]TU Wien, Vienna, Austria

Abstract

Inspection drones can run small vision models on board and send images to larger models in the cloud. The drone must choose models for each image, with limited time and a weak radio link. We show that a small language model can learn to make this choice for power-line inspection. Step by step, the model picks one of about 60 vision tools and reads its answer. At the end it writes a report with three parts: the component type, whether there is a defect, and the condition. We train this agent by supervised fine-tuning and reinforcement learning and compare its reports with the true labels. We report four results. First, supervised fine-tuning turns a model that writes no correct report into an agent with 88.1% fully correct reports. Second, the set of tools matters: when we replace one large general model with a small component classifier, the agent learns to use two small classifiers on the drone, and 95.9% of its reports are fully correct, in 0.09 seconds per image and with no upload. Third, reinforcement learning with a reward on the final report raises fully correct reports from 79.8% to 88.4% and the share of defects found from 68% to 83%. Fourth, reinforcement learning helps the agents that we tested to adapt: when the general model is removed at test time, an agent with 1.7 billion parameters uses a tool that it never saw in training, and 92.4% of its reports are fully correct, against 76.2% without reinforcement learning.

Keywords: adaptive AI inference, edge-cloud continuum, resource-efficient language models, onboard processing, distributed intelligence, autonomous inspection, tool selection

1. Introduction

Drones are changing how power lines, bridges and towers are inspected. A drone takes many detailed images during a flight, and new drones can also analyse these images on board [1]. For this, a drone can use many AI models. We call them *tools*. Some are small classifiers on the drone. Some are vision-language models (VLMs) on the drone. Others are large models in the cloud, and to use them the drone must upload the image over a radio link that is often slow or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson board and need 1.4–6.4 s for each video segment [2]. Azimi et al. compare three ways to run such a model: on the drone, in the cloud, or split between the two [3]. Both groups say that an adaptive controller, which decides during the flight which tool to use, is future work.

Language models that use tools are now common [8, 9, 10]. Zhou et al. [5] show that even small language models can learn to select tools from a long list. They train the model first with supervised fine-tuning (SFT) and then with a reinforcement learning method called GRPO [6]. A small model of this kind could run next to the other software of the drone and adapt the inference to each image, across the drone and the cloud.

This article asks two questions: *can a small language model learn to choose the tools of an inspection drone, and what does reinforcement learning add?* We build a set of about 60 vision tools and save their answers for every image. Then we let Qwen3 language models [7] with 0.6 and 1.7 billion parameters work as agents: they call tools step by step and then write a report. We train them with SFT and GRPO, and we compare them with the same models without training and with each tool alone. We also remove tools at test time to see if the agents can adapt.

Our main contributions are:

- a tool-selection benchmark for drone inspection with 62 real vision tools on the drone and in the cloud, measured run times, recorded mobile-network data and an upload cost;
- a small agent that learns the task with SFT: fully correct reports rise from 0% to 88.1%;
- evidence that a good tool set makes the agent accurate and cheap: with a small component classifier, 95.9% of the reports are fully correct, in 0.09 s per image and with no upload;
- GRPO on full inspection episodes with a reward on the final report, which raises fully correct reports from 79.8% to 88.4% and the share of defects found from 68% to 83%;
- evidence that GRPO helps the tested agents to adapt to a changed tool set: without new training, a 1.7B agent uses a

*Corresponding author.
Email address: mahdi.khashan@student.tuwien.ac.at (Mahdi Khashan)

tool that it never saw in training and reaches 92.4%, against 76.2% without GRPO.

Section 2 reviews related work. Section 3 describes the data, the tools, the agent and the training. Section 4 presents and discusses the results, and Section 5 concludes.

2. Related Work

Edge VLMs for inspection. EVLM [2] runs a SmolVLM-256M with LoRA on a Jetson and reduces the upload of a 4 s video segment from 485 kB to 25 kB. The study in [3] measures SmolVLM on the drone, in the cloud, and split between them, for different link speeds. INAR-VL [4] sends each request to the edge or to the cloud, depending on the input and the link speed. Farahani et al. [1] show that SmolVLM-256 needs about 2.8 s per image on a Jetson Orin, while a YOLO detector needs 0.14 s. They conclude that a light model should start the VLM only when it is needed. All these systems use fixed rules to decide where one model runs. They do not choose, for each image, among many models with different strengths, costs and locations.

Language models that use tools. Toolformer [8] and ReAct [9] let a language model call tools inside its own text. ToolLLM [10] and Gorilla [11] train models on thousands of software interfaces and use a search step to shorten the list of tools. Zhou et al. [5] split the problem into planning, search and selection. They show that small models trained with SFT and GRPO can also use new tools. Other work shows that a small model becomes better when it learns from the written reasoning of a larger model [12]. These studies use software tools with no time limit, no network and no real cost per call. Their tools rarely disagree, and success usually means a correct call, not a correct final answer.

3. Methodology

We first describe the data, the tools and what a tool call costs. Then we define the report that the agent must write and how the agent works step by step. Finally we define how the agent learns: with SFT, with GRPO on single steps, and with GRPO on full episodes.

3.1. Data

We use the component images of InsPLAD [23] with the `tower14` split of ElecVQA-Bench [22]. The 18 training towers are used to train the tools. The 10 validation towers (5,624 images) are used to build the training data of the agent. The 14 test towers (6,092 images) are used for testing. There are 11 component types and six conditions: good, rust, corrosion, missing cap, bird nest and other defect. Only 9.9% of the test images show a defect.

3.2. Tools

Table 1 shows the 62 tools in groups. We fine-tuned the classifiers on the training towers. We ran every tool once on every validation and test image and saved the answers. During training and testing, the agent gets these saved answers. The three laya-vision tools [18] answer all three questions of the report: component, defect and condition. A ResNet-18 component classifier answers only the component; we added it after the first experiment (Section 4.3), so the first experiment uses the other 61 tools. Fourteen tools run on the drone, for example ResNet-18, DeiT-S, MobileNet, the component classifier, a compressed laya-vision and SmolVLM. The other 48 tools run in the cloud.

Table 1. The 62 tools in groups. Answer: the report fields that the tool can answer (C: component, D: defect, S: condition).

Group	Examples	Tools	Answer
Image classifiers	ResNet-18/50/101 [19], ConvNeXt, ViT, DeiT, Swin, EfficientNet, MobileNet, YOLO	12	D, S
Classifier variants	mirror image, five crops, new cut from the full photo	6	D, S
Component specialists	ResNet-50 and ConvNeXt for each component group	10	D, S
Feature classifiers	DINOv2 [20], CLIP [21]: condition classifier and defect detectors	15	D or S
Zero-shot	CLIP-L with text prompts	1	D, S
Small VLMs	SmolVLM-256M [17], 3 variants	3	D
laya-vision	full, compressed (4-bit), new cut	3	C, D, S
Large VLM	Qwen3-VL-8B, 8 questions or cuts	8	D
Component classifier	ResNet-18 on 11 component types	1	C
Image checks	quality, exposure, size	3	–

3.3. Time, network and limits

Each tool call costs time. We measured the run time of each tool on a desktop GPU. We have no Jetson board, so we multiply the time of a drone tool by 9. This is our estimate of how much slower a Jetson Orin NX is.

A cloud tool needs its run time, 0.05 s for the network round trip, and the time to upload the image. The upload time is the image size divided by the link speed. The drone uploads each image only once, so later cloud tools for the same image do not pay the upload again. The link speed follows recorded 4G/LTE data [24], reduced to 10% (about 3 Mbit/s). Sometimes the link is down, and then no cloud tool can be called. A typical image has 33 kB.

The agent has a time budget of 4, 10 or 30 s for each image. A tool can be called only if it was not called before and its time fits in the time that is left. The agent has at most 8 steps; at step 8 it must write the report.

3.4. The report

For each image, the agent must answer three questions:

1. *Which power line component is in this image?* One of 11



Figure 1. **Three test images, without and with tool answers.** (a) The image alone, as the drone takes it. The agent never sees it. (b) The true labels and the answers of two tools, which is all the agent reads. Orange: a wrong answer. In the second image, ResNet-50 is unsure and wrong, while laya-vision finds the missing cap. In the third image, laya-vision names the wrong component but the right condition.

types, for example glass insulator, shackle or yoke suspension.
2. *Does this power line component have a defect?* Yes or no.
3. *Which condition does this power line component have?* One of six conditions: good, rust, corrosion, missing cap, bird nest or other defect.

The agent gives the answers as one short JSON object, for example

```
{"name": "REPORT", "component": "glass insulator",
   "defect": "yes", "condition": "missing cap"}
```

A field is correct only if it is exactly equal to the true label. Figure 1 shows three test images with their true labels and the answers of two tools.

3.5. The agent: plan, select and report

The agent is a text-only language model. It never sees the image. It only reads the answers of the tools as short text. For each image it repeats three stages until it writes the report.

Plan. The agent reads the state: the task, the time left, the state of the link, whether the image is already uploaded, and the tool answers so far. The thinking agents (Section 3.8) then write a short reason, which works as a plan for the next action.

Select. The agent receives a list of 15 tools that can still be called, plus the option *report*. It answers, for example, `{"name": "SELECT", "ids": ["resnet18"]}`. The saved answer of this tool is then added to the next prompt as one line of text, such as *"resnet18: missing cap 99%, good 1%"*.

Report. When the agent thinks it knows enough, it chooses *report* and writes the three answers (Section 3.4). The work on this image then ends.

Ranking of the tools. With 62 tools, a full list would fill the prompt, so we show only 15. We rank the tools with BM25 [13], a standard text-search method. Each tool has a short description: its name, its location (drone or cloud) and one line about what it does. The search text is a fixed sentence about the task plus the answers of the tools called so far. BM25 gives a high score to a tool whose description shares many words with the search text, and rare words count more than common words. We show the 15 tools with the highest score. Because the search text contains the tool answers, the list changes during the work on an image: after the answer "missing cap 82%", for example, the tools that check only for a missing cap move to the top.

3.6. Training with SFT

Like Zhou et al. [5], we first train each step separately. For each step we need a *target*: the action that we want the agent to take.

Target for a tool step. We look at the true labels of the image and ask which tool would add the most correct report fields that are still missing. This tool is the target. If two tools add the same number of fields, the faster tool is the target. If no tool can add a correct field, the target is to write the report.

Target for a report step. The target is the report with the true labels. So the agent must learn which tool to trust when two tools disagree.

We built $N = 61{,}461$ training steps from the validation images, each with a state s_i (the prompt) and a target action a_i. SFT teaches the agent to give a high probability $p(a_i \mid s_i)$ to the target. It minimises the loss

$$L_{\mathrm{SFT}} = -\frac{1}{N}\sum_{i=1}^{N} \log p(a_i \mid s_i). \tag{1}$$

The loss is small when the agent is sure of the target action in every training step. We train LoRA adapters [16] on 12,000 of these steps with the TRL library [14]. The *base* model is the same language model with the same instructions but without training.

3.7. Training with GRPO on single steps

GRPO [6] is a reinforcement learning method. It starts from the SFT agent. For one training step, the agent writes 8 different answers, and each answer gets a reward:

$$R = \begin{cases} 1 & \text{the target tool is selected} \\ \dfrac{\text{correct fields}}{3} & \text{a report, if the target is the report} \\ 0 & \text{in all other cases.} \end{cases} \tag{2}$$

GRPO then compares the 8 answers. Let R_j be the reward of answer j, $\bar{R}$ the mean of the 8 rewards and σ their standard deviation. The loss is

$$L_{\mathrm{GRPO}} = -\frac{1}{8}\sum_{j=1}^{8} \frac{R_j - \bar{R}}{\sigma} \log p(\mathrm{answer}_j \mid s) \; + \; \beta\, D_{\mathrm{KL}}. \tag{3}$$

The first part makes answers with a reward above the mean more likely and answers below the mean less likely. In the second part, D_{KL} is the Kullback–Leibler divergence. It measures how different the new agent is from the SFT agent, and the weight $\beta = 0.02$ keeps the new agent close to the SFT agent. Equation (3) is a simplified form: the library also limits how far one update can move the agent, as in PPO [15]. We train for 400 steps on 4,000 states.

3.8. Teacher reasoning (thinking agents)

A reason that is written before an action can help a small model. We asked a larger model, Claude Haiku 5.5, to write a short reason (80–150 words) for 4,000 training steps. The reason explains the state, the missing fields, the time and the link, and why the target action is a good choice. It does not mention the true labels. 3,780 reasons passed our automatic checks; Box 1 shows one of them. SFT then teaches the agent to write a reason before each action, and GRPO continues with the same reward. The reward looks only at the action, not at the reason. We call these agents *thinking* agents. The component classifier did not exist when we trained them, so it is a new tool for them.

Box 1: A teacher reason from the training data.

State: no time limit, 0.80 s used; link up, 2.6 Mbit/s; image already uploaded.
Tool answers so far:
`efficientnet: missing cap 100%, corrosion 0%`
`laya: component glass insulator 88%; defect 97%;`
`missing cap 98%, good 1%`

Reason (Haiku 5.5): "I have used 0.80 s with no time limit, and the image is uploaded at 2.6 Mbit/s. Laya reports glass insulator at 88%, with defect at 97% and missing cap at 98%, and good, no defect at only 1%. Efficientnet reports missing cap at 100% and corrosion at 0%. Two different models agree on missing cap, and the defect reading is high. The condition is clear, so I stop and report glass insulator with a defect and condition missing cap."

Action: `{"name": "REPORT", "component": "glass insulator", "defect": "yes", "condition": "missing cap"}`

3.9. Training with GRPO on full episodes

The reward in Eq. (2) looks at one step. What we really want is a correct final report. We therefore also train with a reward on the whole inspection of an image, which we call an *episode*. In an episode, the agent works in one conversation: it writes a reason, selects a tool, reads the result, and so on, until it writes the report. For one image, the agent plays 8 episodes. Each episode gets one reward at the end:

$$R_{\text{episode}} = \frac{\text{correct fields in the report}}{3} - 0.01 \times T, \quad (4)$$

where T is the total time of the tool calls in seconds. A correct report earns almost 1, and a slow plan earns a little less. There is no target tool: every plan that leads to a correct report earns the same reward. The loss is Eq. (3), with the 8 episodes in place of the 8 answers. Only the text that the agent wrote is used for learning; the tool results are not. We start from the 1.7B thinking SFT agent and train for 150 steps, with 2 images and 8 episodes per step.

Table 2. Accuracy of single tools on all 6,092 test images. "–": the tool does not answer this field. Best results in bold.

Tool	Runs on	Component	Defect	Condition
laya-vision	cloud	0.924	0.947	0.952
laya-vision, compressed	drone	0.870	0.944	0.938
ResNet-18	drone	–	0.981	0.979
DeiT-S	drone	–	0.982	0.980
ConvNeXt-T, new cut	cloud	–	**0.988**	**0.988**
Component ResNet-18	drone	**0.996**	–	–

4. Experimental Evaluation

We test the agents without thinking on 1,500 of the 6,092 test images, chosen with a fixed random seed. We test the thinking agents, and the agents with a changed tool set, on the first 500 of these images. We count how often each report field is correct and how often all three fields are correct. All tests use one seed, and the agent always takes its most likely answer. With 500 images, a difference of about three points can be chance.

We first look at the tools alone and at the agent after SFT. Then we change the tool set, train with a reward on the final report, and test how the agents adapt when a tool is removed.

4.1. The tools alone

Table 2 shows how accurate some tools are when each one answers alone. The fine-tuned classifiers are strong on defect and condition: a ConvNeXt on a new cut of the component is correct on 98.8% of the images. Laya-vision answers all three fields and names the right component on 92.4% of the images. The component classifier is correct on 99.6%.

4.2. SFT teaches the agent the task

Table 3 and Figure 2 show the accuracy of the agent's reports with the first 61 tools. Without training, Qwen3-0.6B never calls a tool. It writes a report at the first step and always says "no defect", so it finds no defect at all, and no report is fully correct.

After SFT, 88.1% of the reports are fully correct, and the agent finds 88.5% of the defects. It calls one tool for each image and then writes the report. On 95% of the images the tool is laya-vision in the cloud. On the other images it is the compressed laya-vision on the drone, mostly when the link is down. So the agent has learned to move the work to the drone when the network fails. One call needs about 0.35 s, far below the smallest budget of 4 s.

The agent is also a little better than the tool it uses. Its component accuracy is the same as that of laya-vision (0.92), but its defect and condition answers are better (0.961 against 0.947, and 0.958 against 0.952). The agent has learned to correct some answers of laya-vision when it writes the report.

4.3. A small specialist instead of a large general model

Laya-vision is a large general model in the cloud. Can small tools on the drone do the same work? We added the ResNet-18 component classifier as a drone tool. It answers only the component, is correct on 99.6% of the test images (Table 2) and needs

Table 3. Report accuracy of the Qwen3-0.6B agent on 1,500 test images, single seed. All three: share of reports where component, defect and condition are all correct. Time and upload: mean per image. Best results in bold.

Method	Budget	Component	Defect	Condition	All three	Defects found	Time	Upload
Base (no training)	4 s	0.037	0.895	0.000	0.000	0.000	0.00 s	0.04 kB
SFT	4 s	**0.919**	**0.961**	**0.958**	**0.881**	0.885	0.35 s	34.1 kB
SFT	10 s	0.917	**0.961**	**0.958**	0.880	**0.892**	0.37 s	34.1 kB
SFT	30 s	0.915	0.959	0.957	0.876	0.885	0.41 s	33.7 kB

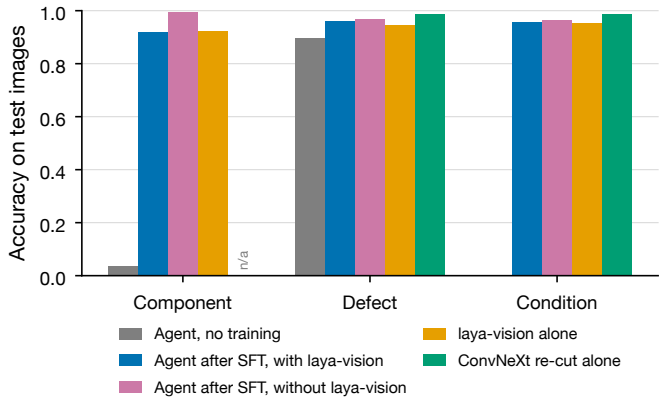


Figure 2. **The tool set decides what the agent can reach.** Accuracy for each report field at the 4 s budget. With laya-vision, the trained agent is as good as laya-vision. Without it, the agent uses two classifiers on the drone and is better on every field. Agents: 1,500 test images; single tools: all 6,092 test images; single seed. ConvNeXt re-cut does not answer the component (n/a).

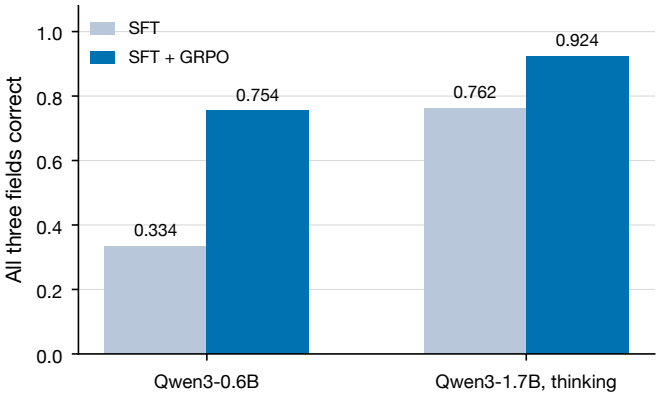


Figure 3. **GRPO helps the agents adapt when the general model is removed.** Fully correct reports at the 4 s budget. The agents were trained with laya-vision; it is removed at test time, with no new training. 500 test images, single seed.

only 2.8 ms on a desktop GPU. Then we removed the three laya-vision tools, built the training data again with the same target rule, and trained the agent in the same way.

Table 4 shows the result. The agent now calls two tools for every image: the component classifier, and ResNet-50 (95%) or ResNet-18 (5%) for defect and condition. All of them run on the drone, so nothing is uploaded. Fully correct reports rise from 88.1% to 95.9%, and the time per image falls from 0.35 s to 0.09 s. The agent is better on every field (Figure 2).

4.4. A reward on the final report

Table 5 compares three 1.7B thinking agents with all 62 tools: after SFT, after GRPO on single steps (Section 3.7), and after GRPO on full episodes (Section 3.9). All three are tested in the conversation format of Section 3.9, on the same 500 images at the 4 s budget.

Both kinds of GRPO improve the agent, and the reward on the final report gives the best agent. Fully correct reports rise from 79.8% after SFT to 84.2% with GRPO on single steps and to 88.4% with GRPO on full episodes. The full-episode agent always writes a valid report, while the other two agents fail to do so on about 4% of the images. It also finds 83% of the defects, against 68–69% for the other two agents. This matters for inspection, because a missed defect is the most expensive error. During training, the mean episode reward rose from 0.85 to 0.91.

4.5. Adapting to a changed tool set

In a real drone fleet, tools will be added, removed and replaced. So we tested what happens when the tool set changes after training. We took agents that were trained with laya-vision and removed the three laya-vision tools at test time. We did not train the agents again. To write a correct report, an agent must now find other tools. One of them is the component classifier, which was never a target in its training data.

Figure 3 and Table 6 show the result for two pairs of agents. In each pair, the GRPO agent started from the SFT agent, and both were trained with the single-step reward. In both pairs, the GRPO agent adapts much better. The 0.6B agent with GRPO reaches 0.754, against 0.334 for its SFT agent, and it calls the component classifier on 72% of the images. The 1.7B thinking agent with GRPO reaches 0.924, against 0.762. This is close to the agent that we trained again without laya-vision (0.959). It calls the component classifier on almost every image and then ResNet-50, which is the same plan as that of the newly trained agent. So this agent uses a new tool, which it knows only from its description, without any new training.

4.6. Time and upload

Table 7 compares the time and upload of our agents with published measurements on a Jetson. The numbers are not fully comparable: an EVLM segment has several video frames and a text report, while we process one image and give three answers. Our times include only the tools, not the agent itself.

Table 4. The same training with a different tool set. Qwen3-0.6B agent at the 4 s budget, 1,500 test images, single seed. Without laya-vision, the ResNet-18 component classifier is in the tool set. Calls: tools called per image. Best results in bold.

Tools	Method	Component	Defect	Condition	All three	Defects found	Time	Upload	Calls
With laya-vision	SFT	0.919	0.961	0.958	0.881	**0.885**	0.35 s	34.1 kB	1.0
Without laya-vision	base	0.019	0.346	0.003	0.001	0.452	0.14 s	19.2 kB	0.4
Without laya-vision	SFT	**0.994**	**0.967**	**0.965**	**0.959**	0.879	**0.09 s**	**0 kB**	2.0

Table 5. GRPO on full episodes. Qwen3-1.7B thinking agents with all 62 tools, conversation format, 4 s budget, 500 test images, single seed. Not valid: share of images without a valid report. Best results in bold.

Agent	All three	Defects found	Not valid	Time
SFT	0.798	0.678	3.8%	0.44 s
GRPO on single steps	0.842	0.695	4.0%	0.37 s
GRPO on full episodes	**0.884**	**0.831**	**0%**	0.37 s

Table 6. Laya-vision removed at test time, 4 s budget, 500 test images, single seed. New tool: share of images where the agent calls the component classifier, which was never a target in its training. Last row: agent trained again without laya-vision (1,500 images). Best results of the adapted agents in bold.

Model	Method	Comp.	Cond.	All three	New tool
Qwen3-0.6B	SFT	0.348	0.898	0.334	0.00
	SFT + GRPO	0.774	**0.956**	0.754	0.72
Qwen3-1.7B	SFT, thinking	0.830	0.846	0.762	0.84
	SFT + GRPO, thinking	**0.988**	0.942	**0.924**	**1.00**
Trained again without laya-vision		0.994	0.965	0.959	1.00

Table 7. Time and upload. Other work: measured on a Jetson Orin. This work: mean per test image at the 4 s budget, time of the tools only.

System	Time	Upload
Other work		
YOLO detector, per image [1]	0.14 s	–
SmolVLM-256, per image [1]	2.8 s	–
EVLM (SmolVLM), per 4 s segment [2]	1.4–6.4 s	25 kB
SmolVLM on the drone, per request [3]	2.7–2.8 s	0
This work		
SFT agent (one laya-vision call)	0.35 s	34.1 kB
SFT agent without laya-vision (two drone classifiers)	0.09 s	0
ResNet-18 on the drone, alone	0.03 s	0

4.7. Discussion

A small agent is enough to learn the task. A model with 0.6 billion parameters learns the answer format, a useful tool plan and a careful reading of the tool answers from 12,000 training steps. It also learns to use a drone tool when the cloud link is down.

Good tools make the agent accurate and cheap. The same training gave 7.8 more points of fully correct reports when we replaced one general model with a small specialist, and it removed the upload. So it is at least as important to design and measure the tools as to train the agent. For onboard processing, two small classifiers on the drone were better than one large model in the cloud.

A reward on the final report helps most. GRPO on full episodes does not copy a target tool. It rewards what the operator needs: a correct report. This gave the most accurate agent, the most defects found and no invalid reports.

Reinforcement learning helps the agents adapt. In our tests, the GRPO agents handled a changed tool set much better than their SFT agents, and one of them used an unseen tool almost as well as a newly trained agent. For a drone fleet whose tools change over time, this is a useful property.

Limitations. The agent works with saved tool answers. The network and the times on the drone are simulated, and the factor of 9 is an estimate. The time of the agent itself is not counted. The budgets of 4 s or more never limited the agent, so we could not study the choice between accuracy and time. The thinking agents and the tests with a changed tool set use 500 images. The adaptation results come from two pairs of agents; first tests with other models and rewards show that the effect is not the same for every setting, so it needs a wider study. All results come from one seed and one data set of power-line components.

5. Conclusion

A small language model can learn to inspect power-line images by calling vision tools on the drone and in the cloud. After SFT, a Qwen3-0.6B agent writes a fully correct report for 88% of the images. With a small component classifier in the tool set, the agent uses two classifiers on the drone and reaches 96% in 0.09 s per image, with no upload. GRPO with a reward on the final report gives the most accurate 1.7B agent with all tools: 88% fully correct reports, against 80% after SFT, and 83% of the defects found. When the general model is removed at test time, the GRPO agents that we tested adapt much better than their SFT agents, and a 1.7B agent uses a tool it never saw in training and reaches 92%.

This article is a first report of ongoing work. The evaluation of a larger 4B agent, of more random seeds, of other rewards for full episodes, and of how full-episode agents adapt to a changed tool set is not complete, and we will add these results in the next version. We also plan to train with randomly removed tools, to test tighter time budgets, and to measure the agent and the drone tools on a Jetson board.

Data and code availability

The code, the saved tool answers and the result files are in the project repository, branch `agentic-ppo-v2` (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [22, 23]. The laya-vision weights are published under CC BY-NC-SA 4.0 [18].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[5] Zhou et al., Training small language models for transferable tool selection, arXiv:2610.31483 (2026).

[6] Z. Shao, et al., DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, arXiv:2402.03300 (2024).

[7] A. Yang, et al., Qwen3 technical report, arXiv:2505.09388 (2025).

[8] T. Schick, et al., Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[9] S. Yao, et al., ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[10] Y. Qin, et al., ToolLLM: Facilitating large language models to master 16000+ real-world APIs, in: International Conference on Learning Representations (ICLR), 2024.

[11] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive APIs, arXiv:2305.15334 (2023).

[12] C.-Y. Hsieh, et al., Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes, in: Findings of the Association for Computational Linguistics (ACL), 2023.

[13] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.

[14] L. von Werra, et al., TRL: Transformer reinforcement learning, software (2020). https://github.com/huggingface/trl.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).

[16] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[17] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[18] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] M. Oquab, et al., DINOv2: Learning robust visual features without supervision, Transactions on Machine Learning Research (2024).

[21] A. Radford, et al., Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning (ICML), 2021.

[22] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[23] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[24] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.